norma

Installation & Usage Guide

Learn the denial constraints that govern a table, detect data-quality errors, and normalize it.

Engine: **CPAD** (Constrained Predicates for Anomaly Detection)

Apache-2.0 · github.com/normaCPAD/norma

Contents

1 Overview

NORMA discovers the functional dependencies (FD) and denial constraints (DC) that a single, mostly clean relational table satisfies, scores each cell by its degree of *violation* (the CPAD engine), and derives candidate keys, the current normal form, and a 3NF/BCNF decomposition. Unlike classical anomaly detectors, it separates a **relational error** (a value that breaks a constraint) from a **rare-but-valid** value. It then exports the discovered constraints to the data-quality tools you already use.

Three ways to use it: a command-line tool (`norma`), a Python library (`import norma`), and an interactive desktop application (`norma studio`).

2 Installation

2.1 From PyPI

```
pip install norma-dq # core (the import name stays: import norma)
pip install "norma-dq[gated]" # + PyTorch for the differentiable GatedCPAD variant
pip install "norma-dq[studio]" # + PySide6 for the desktop application
pip install "norma-dq[lake]" # + Parquet / DuckDB / Excel ingestion
pip install "norma-dq[baselines]" # + pyod (Isolation-Forest baseline for `norma bench`)
```

The distribution is named `norma-dq` on PyPI, but the Python package you import is always `norma`.

2.2 From source

```
git clone https://github.com/normaCPAD/norma.git
cd norma
pip install -e ".[studio,lake,baselines,dev]"
pytest -q # run the test suite
```

2.3 Requirements

Python $\geq$ 3.9. Core dependencies (NumPy, pandas, SciPy, scikit-learn, PyYAML) are installed automatically. The differentiable variant and the desktop app are optional extras.

3 Quick start (command line)

Given any tabular file, discover its constraints and a proposed data model:

```
norma analyze data.csv
```

This prints the discovered constraints (with confidence), the candidate keys, the current normal form, a proposed 3NF/BCNF decomposition, and the worst constraint-violating cells.

4 Command-line reference

4.1 `norma analyze`

```
norma analyze PATH [--learner routed|discrete|gated|ensemble]
                   [--tau 0.90] [--max-lhs 2] [--cap 300]
                   [--top 10] [--json model.json]
```

- `-learner` – constraint learner; `routed` (default) is the complete CPAD.
- `-tau` – minimum FD confidence for the discrete learner (`routed`/`gated` self-select it).
- `-max-lhs` – maximum left-hand-side size for composite FDs.
- `-cap` – maximum column cardinality treated as a modeling column.
- `-top` – number of violating cells to display.
- `-json` – also write the full model as JSON.

4.2 `norma export` – to your data-quality stack

```
norma export PATH --to FORMAT [--out DIR] [--learner ...] [--tau ...]
```

`-to`	Output
`great_expectations`	Expectation Suite (keys → compound-unique; FDs → row checks)
`pandera`	`DataFrameSchema` with group-wise FD and linear checks
`dbt`	`schema.yml` + one singular test (SQL) per dependency
`sql_check`	ANSI `UNIQUE`/`CHECK` + violation-monitor views
`json_schema`	per-record JSON Schema (types / required)
`frictionless`	Table Schema (types + `primaryKey`/`uniqueKeys`)
`metanome`	FD/DC lines in Metanome result syntax
`shacl`	SHACL shapes with `sh:sparql` (knowledge-graph stack)

NORMA discovers the rules; your pipeline enforces them.

4.3 `norma freeze` / `norma check` – contracts & CI

Freeze the accepted constraints to a versioned `norma.yml` that lives in git, then re-validate any table against it in continuous integration:

```
norma freeze data.csv -o norma.yml
norma check data.csv -c norma.yml --fail-on-violations
```

`check` exits with status `1` when any constraint is violated, so it gates a pipeline directly. A discovered *candidate key* is validated by the robust *determinant* property (it determines every column), not by flat-table row uniqueness.

4.4 `norma score` – score the whole dataset

Apply the learned rules to *every* row of a (possibly huge) file, streaming it in chunks so memory stays bounded. Without `-max-rows`, the functional dependencies are discovered on the *entire* file (streaming, no sample); with it, they are learned on a sample (which also enables composite and numeric constraints) and every row is still scored.

```
norma score data.csv # learn FDs on the WHOLE file (streaming, no sample), score all
norma score data.csv --out scored.csv # also write norma_score / norma_rule columns
norma score data.csv --max-rows 200000 # learn on a sample (composite/numeric FDs), score all
```

With `-out`, the data is written back with two columns: `norma_score` (the per-row violation degree) and `norma_rule` (the responsible constraint).

4.5 `norma bench` – reproducible evaluation

Evaluate detection on aligned clean/dirty pairs (the Raha/Baran layout) and report AUROC/AUPRC against the ground-truth error mask, next to an Isolation-Forest baseline:

```
norma bench --data ./datasets --learner routed --json results.json
```

Either dataset layout is discovered automatically under `-data`:

```
datasets/<name>/clean.csv datasets/<name>/dirty.csv
datasets/<name>_clean.csv datasets/<name>_dirty.csv
```

5 Python & notebook API

```python
from norma.core.table import Table
from norma.models import RoutedCPAD
from norma.modeling.report import build_model
from norma.export import export

t = Table.from_csv("data.csv") # or from_parquet / from_excel / from_json / from_any
report = build_model(t, RoutedCPAD().fit(t))
print(report.to_text())

files = export(report, "great_expectations") # {filename: content}
```

In Jupyter/Colab, `t.profile()` renders an HTML report inline whose top violations are colour-coded: constraint violation versus rare-but-valid.

```python
from norma.core.table import Table
Table.from_csv("data.csv").profile()
```

6 The desktop application (norma studio)

Launch the interactive application:

```
norma-studio # or: python -m norma.studio
```

Features. Open a CSV/Parquet/Excel/JSON file or connect to a relational database; discover FD, composite, order and linear constraints; enable/disable them and add expert rules (the schema re-synthesizes live); interactive FD-graph and relational-schema diagrams; bidirectional visual ↔ SQL; an anomaly heatmap; constraint-guided repair; a one-click data-quality report; and a clean-database builder. The **File** menu also exports the discovered constraints to any ecosystem format, and freezes/checks a `norma.yml` contract. Bilingual (FR/EN), themeable (light/dark).

7 Building a standalone executable

A self-contained desktop bundle is produced with PyInstaller:

```
cd norma
bash packaging/build_linux.sh # -> dist/norma-studio/norma-studio
```

The script encodes three environment fixes; run it (rather than invoking `pyinstaller` directly) so they all apply:

1. **Recursion limit** – the deep NumPy/SciPy/scikit-learn import graph exceeds Python's default; the spec raises it (`sys.setrecursionlimit`).
2. **Kerberos library clash** (conda vs. system) – Qt's `QtNetwork` is imported during analysis; the script preloads a consistent `libkrb5` family.

3. **Symlinks on exFAT/NTFS** – PyInstaller links shared libraries; the script builds on a native filesystem and copies the bundle back symlink-free (`rsync -L`), so it works on external drives.

Windows / macOS. The spec embeds `icon.ico` (Windows) and `icon.icns` (macOS, with a `.app` bundle). Build on the target OS with `pyinstaller packaging/norma-studio.spec`.

7.1 Linux taskbar/dock icon

The window icon is set in-app; the *taskbar* icon is resolved from a desktop entry. Install it once:

```
bash packaging/install_linux_desktop.sh
```

For a packaged build, point the launcher at the bundled binary:

```
sed -i "s|^Exec=.*|Exec=$PWD/dist/norma-studio/norma-studio|" \
  ~/.local/share/applications/norma-studio.desktop
```

8 Troubleshooting

`Spec file ... not found`
You are in the wrong directory. Run from the `norma` project root, where `packaging/` lives.

`RecursionError` during a build
Use `bash packaging/build_linux.sh`; if it persists, raise `sys.setrecursionlimit` in the spec.

`undefined symbol: k5_buf_cstring`
A conda/system Kerberos mismatch when Qt loads `QtNetwork`. Build via the script, or prefix the command with `LD_LIBRARY_PATH=$CONDA_PREFIX/lib`.

`os.symlink ... Operation not permitted`
The output path is on exFAT/NTFS. Build via the script (it copies symlink-free), or target a native filesystem.

Taskbar shows a generic icon
Run `install_linux_desktop.sh` and launch the app via the `norma-studio` command so its window class matches the desktop entry.